

MCSMP20 Monitor Program – Internal Subroutines

by Chuck Yakym and Lee Hart

Last update: September 1, 2023

This note documents the internal subroutines within the MCSMP20 series of Monitor Programs for the 1802 microprocessor. The program is offered in a number of different versions, for various 1802 computers with different memory maps.

Different versions of the program have been assembled to run at org.0 or org.8000h. Org.0 versions expect RAM to be at 8000-FFFFh, while org.8000h versions expect RAM at 0-7FFFh. Because of this, there are two sets of entry point addresses. Use the ones that correspond to your system's memory map.

MCSMP20 uses the 1802 Q pin for serial output, with the data either true, or inverted. They use the EF3 pin for serial input. On power-up or Clear, the program checks the logic level of the EF3 pin, assumes that it is at the “idle” level, and sets the “idle” level of Q accordingly (but Q may be true or inverted, depending on the version of MCSMP20).

The first character received should be an ASCII <CR> (Carriage Return; the “Enter” key on a PC keyboard). It is used to determine the baud rate, which can generally be from 1200-9600 baud with a 1.8 MHz or 4 MHz 1802 clock frequency.

Subroutines	Org. 0000h	Org. 8000h
SCRT CALL	0ADBh	8ADBh
SCRT RETURN	0AEDh	8AEDh
INPUT	0005h	8005h
OUTPUT	021Dh	821Dh
HEXINPUT	00A3h	80A3h
OUTSTR	0526h	8526h
HEXCRLF	0519h	8519h
BINASC	05BFh	85BFh
ASCBIN	053Bh	853Bh
WENTRY	0B5Eh	8B5Eh

User Programs	Org. 0000h	Org. 8000h
RUN	1680h	9680h
HELLO WORLD	1660h	9660h

MCSMP20 Subroutines available to the User

SCRT CALL:

The 1802 has no built-in CALL or RETURN instructions. Instead, you “build” the ones you want to save and restore just the things needed for your application. SCRT is the “Standard” Call and Return Technique for the 1802; but there are in fact many variations. The MCSMP20 CALL routine uses R4 as its dedicated program counter. Register usage:

X = 2
R2 = Stack pointer
R3 = Calling program counter
R4 = CALL routine program counter
R5 = RETURN routine program counter
R6 = Pointer to R3 return address, and arguments passed from calling program
RF.1 = Register to Save and Restore contents of D register
D, DF, and the other registers are not affected

Your Main program must use R3 as its program counter. The CALL instruction is:

```
SEP R4      ; CALL subroutine
DW addr     ; 2-byte addr of Called routine (hi byte 1st)
```

If the subroutine expects parameters to be passed to it from the calling program, they follow the CALL address. For example:

```
SEP R4      ; CALL subroutine
DW x526h    ; address of OUTSTR subroutine
DB "Hi",0   ; the data OUTSTR will output
```

Here is the source code for the CALL routine:

```
EXIT  ←   xADA D3      SEP  R3 ; Exit to called routine at R3
ENTER →   xADB BF      PHI  RF ; save D in RF.1
        xADC E2      SEX  R2 ; set X=2 for stack
        xADD 86      GLO  R6 ; push R6 on stack
        xADE 73      STXD
        xADF 96      GHI  R6 ; low byte, then high byte
        xAE0 73      STXD
        xAE1 83      GLO  R3 ; save R3 in R6
        xAE2 A6      PLO  R6 ; (the RETURN address)
        xAE3 93      GHI  R3
        xAE4 B6      PHI  R6
        xAE5 46      LDA  R6 ; get CALL addr via R6
        xAE6 B3      PHI  R3
        xAE7 46      LDA  R6 ; put it in R3
        xAE8 A3      PLO  R3
        xAE9 9F      GHI  RF ; restore D from RF.1
        xAEA 30DA    BR   DA ; go to EXIT
```

SCRT RETURN:

Again, there many variations of the 1802 “Standard” Call and Return Technique. The MCSMP20 RETURN routine uses R5 as its dedicated program counter. Register usage:

X = 2
R2 = Stack pointer
R3 = Calling program counter
R4 = CALL routine program counter
R5 = RETURN routine program counter
R6 = Pointer to R3 return address, and arguments passed from calling program
RF.1 = Register to Save and Restore contents of D register
D, DF, and the other registers are not affected

Your Subroutine must use R3 as its program counter. CALL will have set R6 to the return address of the program that called it. The RETURN instruction is simply:

```
SEP R5      ; RETURN to the program at R6
```

If the subroutine expects parameters to be passed to it from the calling program, they can be read and then skipped over using R6. Execute as many LDA R6 instructions as there are bytes in the data list supplied by the calling program. For example:

```
LDA R6      ; read a byte from the calling program,  
            ; and increment address  
...        ; (use the byte as needed)  
SEP R5      ; RETURN to the calling at R6
```

Here is the source code for the RETURN routine:

```
EXIT  ←   xAEC D3      SEP  R3 ; return to caller at R3  
ENTER →   xAED BF      PHI  RF ; save D in RF.1  
      xAEE E2      SEX  R2 ; set X=2 for stack  
      xAEF 96      GHI  R6 ; copy R6 into R3  
      xAF0 B3      PHI  R3  
      xAF1 86      GLO  R6  
      xAF2 A3      PLO  R3  
      xAF3 12      INC  R2 ; increment to top of stack  
      xAF4 42      LDA  R2 ; pop R6 from stack  
      xAF5 B6      PHI  R6  
      xAF6 02      LDN  R2 ; high byte, then low byte  
      xAF7 A6      PLO  R6  
      xAF8 9F      GHI  RF ; restore D from RF.1  
      xAF9 30EC     BR   EC ; go to exit
```

INPUT: Serial input routine, Inputted character is returned and stored in RB.0

OUTPUT: Serial output routine, Output character stored in RB.0

HEXINPUT: (alternate INPUT SCRT Call)

Sets user input control value in RB.1 of the inputted character.

Inputted Character is stored in RB.0, RB.1 expresses what was inputted

RB.1 = 00h if <cr> inputted

RB.1 = 01h if <sp> inputted

RB.1 = 02h if <esc> inputted

RB.1 = 03h if "0", "1", "2", "3", "4", "5", "6", "7", "8", OR "9" inputted

RB.1 = 04h if "A", "B", "C", "D", "E", OR "F" inputted

RB.1 = FFh for all other inputted keystrokes

OUTSTR: Output a string of characters pointed to by R7 and terminated by 00h.

HEXCRLF: outputs a CR (0Dh) followed by a LF (0Ah) character

BINASC: outputs the binary value in R7.0 in a 2 digit ASCII form

WENTRY: Return to the Monitor program, there are 3 ways to return to the Monitor:

1. Long Branch to address XB5E
2. Long Branch to the starting address of the Monitor
(0000h or 8000h, to start the Monitor you must press
the <CR> key.)
3. Do a software Interrupt by using the D1 opcode (SEP R1) command

ASCBIN: ASCII to Binary subroutine. RA.1 and RA.0 are used to hold the binary value of up to the last four hexadecimal ASCII user inputs. All non-hexadecimal values inputted are ignored except for a valid user input control values (see HEXINP above). This subroutine can only return to the caller when a ESC, CR or Space key is inputted.

NOTE: All inputs are echoed (outputted) as they are entered except the ESC or CR input. If this subroutine is returned by the Space code, the Space (20h) code will be also outputted.

Pre-loaded User Routines

User can use the Monitor's (R)un command, R1680 or R9680 command to setup R2 (as X and stack pointer), R3 as program Counter, R4 as CALL pointer, and R5 as RETURN pointer and begin execution at 0000H (ORG'ed 8000) or 8000H (ORG'ed 0000H).

NOTE: RE is re-loaded with the original baud rate perimeters.

NOTE: R2 equals XXBFh, where XX equals the last page of user RAM.
XXC0h to XXFFh is the Monitor's reserved memory area, Do Not Use.

Hello World Text: This address contains the ASCII string "Hello World" followed by a CR and LF followed by a 00 Hex value. This can be used to demonstrate the OUTSTR routine.

—

1 Sept 2023: Delete the note about the ASCBIN subroutine, because it only uses RB and RA.